

KAFKA

Instalación, puesta en marcha y primeros pasos

Kafka – Conceptos básicos



¿Qué es Apache Kafka?

- Plataforma de **streaming** distribuida de código abierto
- Desarrollada originalmente por LinkedIn, donada a Apache Software Foundation.
- Diseñada para manejar flujos de datos en tiempo real con alta velocidad
- Almacenamiento duradero de eventos para procesamiento posterior .



Características Principales

- **Alto rendimiento:** Maneja millones de eventos por segundo
- **Escalabilidad horizontal:** Se distribuye en múltiples servidores.
- **Tolerancia a fallos:** Replicación automática de datos.
- **Baja latencia:** Procesamiento en tiempo real.
- **Persistencia:** Los datos se almacenan de forma durable.

Conceptos Fundamentales

Topic (Tema)

- Canal lógico donde se publican los eventos
- Ejemplo: payments, user-activity, system-logs

Partition (Partición)

- División de un topic para paralelización
- Cada partición es un log ordenado

Broker

- Servidor que almacena y sirve los datos
- Kafka funciona como cluster de brokers

Producer (Productor)

- Aplicación que publica eventos a topics
- Envía datos a Kafka

Consumer (Consumidor)

- Aplicación que lee eventos de topics
- Procesa los datos de Kafka

Consumer Group

- Grupo de consumidores que procesan en paralelo
- Balanceo automático de carga

Arquitectura Kafka

- Modelo pub-sub desacoplado
- Persistencia configurable (horas, días, permanente)
- Replicación automática entre brokers.
- **Pub-Sub Desacoplado**
 - **Publishers** (productores) y **subscribers** (consumidores) no se conocen entre sí
 - Los productores publican a topics, no directamente a consumidores
 - Los consumidores leen de topics cuando quieren
 - **Ventaja:** Agregar/quitar productores o consumidores sin afectar al sistema

```
[Producer] -----> [Kafka Cluster] -----> [Consumer]
                        |
                        [Topic A]
                        |
                        ├── Partition 0
                        |
                        ├── Partition 1
                        |
                        └── Partition 2
```

Arquitectura Kafka

Particiones - Cuándo y Cuántas

- **¿Cuándo particionar?**
 - Siempre - Kafka requiere al menos 1 partición por topic
 - Más particiones = más paralelismo
- **¿Cuántas particiones?** Depende de:
 - **Throughput objetivo:** Más particiones = más rendimiento paralelo
 - **Número de consumidores:** Max 1 consumidor por partición
 - **Tamaño del cluster:** No más particiones que brokers disponibles
 - **Latencia vs throughput:** Más particiones pueden aumentar latencia

Consumer Groups - De qué dependen

- **Dependen de:**
 - **Carga de trabajo:** Cuántos mensajes procesar simultáneamente
 - **Tolerancia a fallos:** Más consumidores = mejor redundancia
 - **Particiones disponibles:** Máximo 1 consumidor por partición
- **Lógica de procesamiento:** Si es CPU/IO intensivo, más consumidores
- **Ejemplo:** Topic con 6 particiones puede tener máximo 6 consumidores activos por grupo

Casos de Uso Principales

Messaging

- Sustituto de sistemas tradicionales.
- Mayor rendimiento y tolerancia a fallos

Website Activity Tracking

- Seguimiento de actividad web en tiempo real
- Análisis de comportamiento de usuarios

Metrics & Monitoring

- Agregación de métricas operacionales
- Monitoreo centralizado de aplicaciones

KRaft vs ZooKeeper

KRaft Mode (Recomendado)

- Modo nativo sin dependencias externas
- **Simplificación** de la arquitectura
- **Mejor escalabilidad**

ZooKeeper Mode (Legacy)

- Modo tradicional con ZooKeeper
- En proceso de **deprecación**
- Mayor complejidad operacional

Ecosistema Kafka

Kafka Connect

- Integración con bases de datos y sistemas externos

Kafka Streams

- Librería para procesamiento de streams

Schema Registry

- Gestión de esquemas de datos

Herramientas de Monitoreo

- Kafka Manager, Confluent Control Center

Ventajas vs Sistemas Tradicionales

Característica	Kafka	Sistemas Tradicionales
Rendimiento	Millones msg/sec	Miles msg/sec
Persistencia	Configurable	Limitada
Escalabilidad	Horizontal	Vertical
Latencia	Muy baja	Variable
Tolerancia a fallos	Automática	Manual

Empresas que usan Kafka

- LinkedIn (creador original)
- Netflix - Streaming de contenido
- Uber - Seguimiento en tiempo real
- Spotify - Análisis de reproducción
- Twitter - Procesamiento de tweets
- Airbnb - Análisis de reservas
- Gestamp – Comunicación entre “cerebro” y las distintas fábricas.

Conclusiones

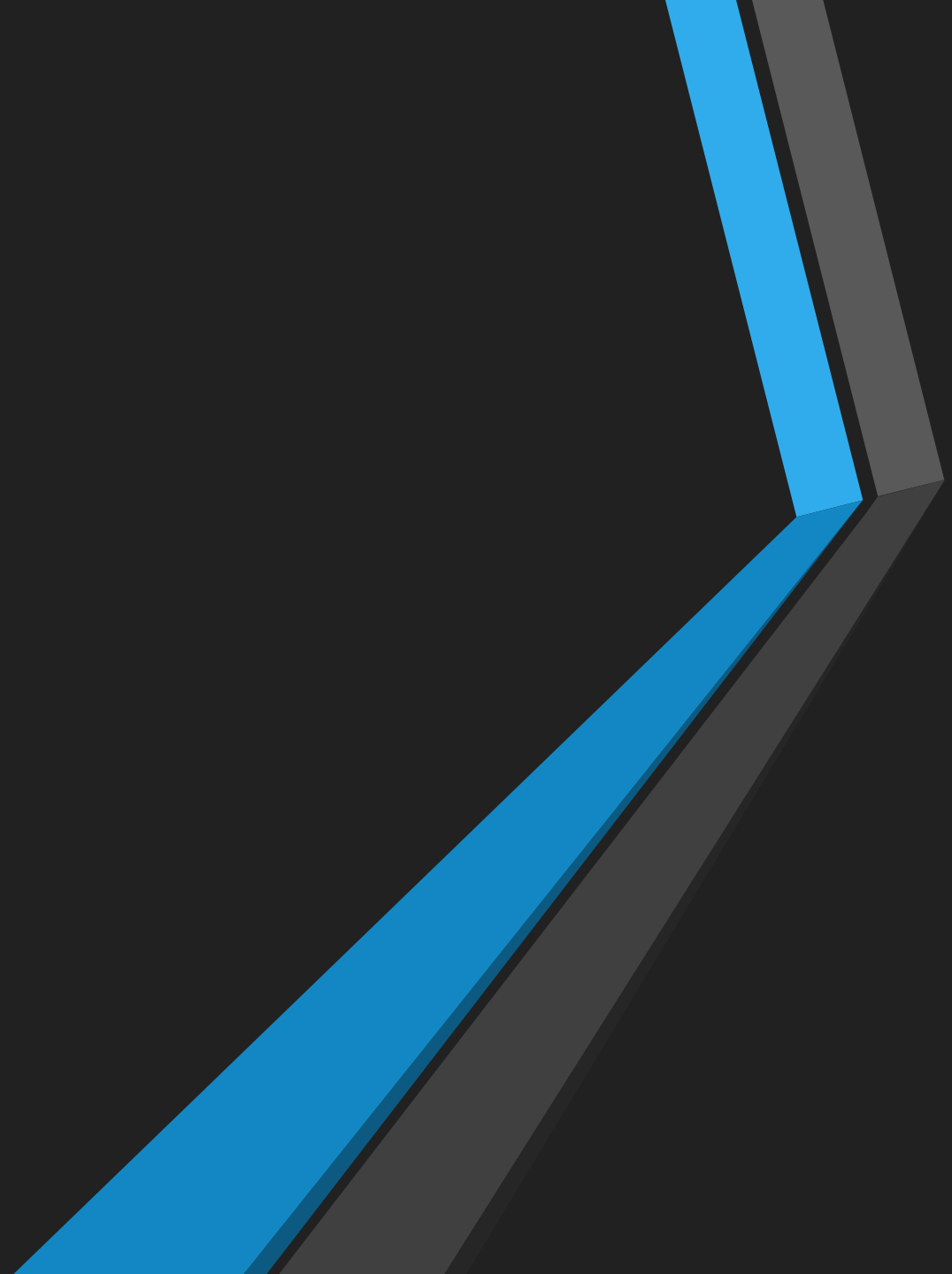
Apache Kafka es ideal para:

- Aplicaciones que requieren **alta velocidad**
- Procesamiento de **grandes volúmenes** de datos
- Arquitecturas de **microservicios**
- **Análisis en tiempo real**

El futuro del streaming de datos

- KRaft simplifica la operación
- Ecosistema rico y en crecimiento
- Estándar de la industria

Kafka – Instalación



Instalación Java

- Asegurarnos que tenemos instalado java v23+ para la versión 4.x de Kafka.
- <https://kafka.apache.org/41/getting-started/compatibility/>

JDK COMPATIBILITY ACROSS KAFKA VERSIONS

Module	Kafka Version	Java 11	Java 17	Java 23
Clients	4.0.0	✓	✓	✓
Streams	4.0.0	✓	✓	✓
Connect	4.0.0	✗	✓	✓
Server	4.0.0	✗	✓	✓

- <https://www.oracle.com/java/technologies/downloads/#jdk25-linux>

Linux	macOS	Windows
Product/file description	File size	Download
ARM64 Compressed Archive	203.25 MB	https://download.oracle.com/java/25/latest/jdk-25_linux-aarch64_bin.tar.gz (sha256)
ARM64 RPM Package	202.81 MB	https://download.oracle.com/java/25/latest/jdk-25_linux-aarch64_bin.rpm (sha256) (OL 9 GPG Key)
x64 Compressed Archive	205.45 MB	https://download.oracle.com/java/25/latest/jdk-25_linux-x64_bin.tar.gz (sha256)
x64 Debian Package	175.91 MB	https://download.oracle.com/java/25/latest/jdk-25_linux-x64_bin.deb (sha256)
x64 RPM Package	204.98 MB	https://download.oracle.com/java/25/latest/jdk-25_linux-x64_bin.rpm (sha256) (OL 9 GPG Key)

```
jam@banano:~/Descargas$ sudo dpkg -i jdk-25_linux-x64_bin.deb
```

Instalación Kafka

- Buscamos en <https://kafka.apache.org/downloads>
- Descargamos la última versión de Kafka (mediante comando o desde la página).
- Creamos una carpeta Kafka y extraemos ahí el contenido.

DOWNLOAD

The project goal is to have 3 releases a year, which means a release every only. It is possible to verify every download by following these [procedures](#).

SUPPORTED RELEASES

4.1.0

- Released September 2, 2025
- [Release Notes](#)
- Docker image: [apache/kafka:4.1.0](#).
- Docker Native image: [apache/kafka-native:4.1.0](#).
- Source download: [kafka-4.1.0-src.tgz](#) (asc, sha512)
- Binary download: [kafka_2.13-4.1.0.tgz](#) (asc, sha512)

```
jam@Dell4:~/kafka$ wget https://dlcdn.apache.org/kafka/3.9.0/kafka_2.13-3.9.0.tgz
```

```
jam@Dell4:~/kafka$ tar -xzf kafka_2.13-3.9.0.tgz
```


Puesta en marcha kafka

- bin/kafka-storage.sh random-uuid `jam@banano:~/kafka/kafka_2.13-4.1.0$ bin/kafka-storage.sh random-uuid`
 - Esto crea un ID para el broker(de manera aleatoria). Lo guardamos para el siguiente comando
- bin/kafka-storage.sh format --standalone -t **FTDy81RNRfCHfGqq-FqfMA** -c config/server.properties
 - Prepara el almacenamiento para el broker Kafka utilizando el broker ID. Sustituir lo que está en negrita por el ID que nos ha devuelto el primer comando. `jam@banano:~/kafka/kafka_2.13-4.1.0$ bin/kafka-storage.sh format --standalone -t mc0SSXA7THSeS5w5u53IXw -c config/server.properties`
- bin/kafka-server-start.sh config/server.properties
 - Lanzamos el servidor.

<https://kafka.apache.org/documentation/#gettingStarted>

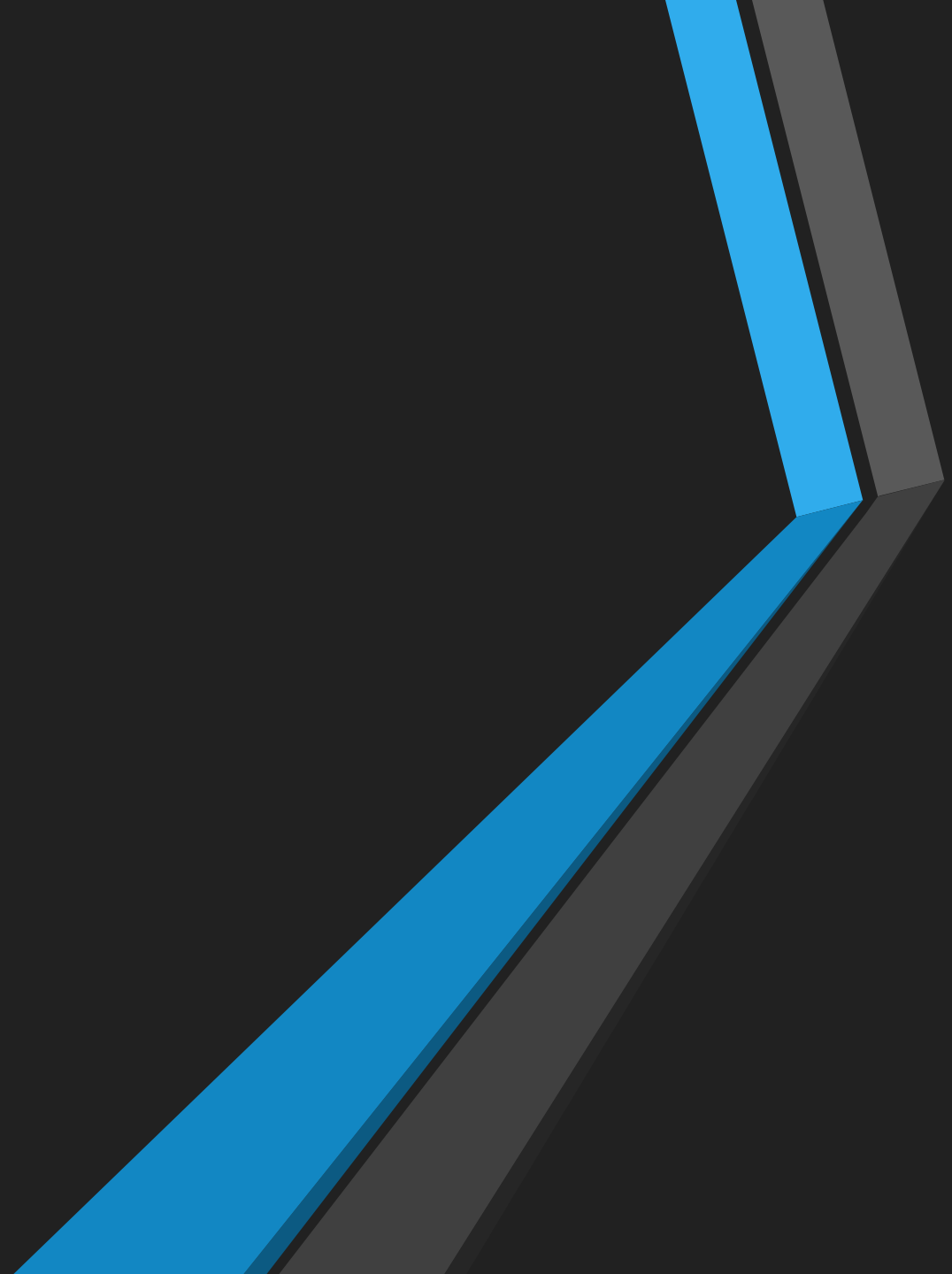
Referencia

Puesta en marcha kafka

- Deberíamos ver el siguiente mensaje en la terminal:

```
[2025-09-22 10:59:43,306] INFO [BrokerServer id=1] Transition from STARTING to S
TARTED (kafka.server.BrokerServer)
[2025-09-22 10:59:43,307] INFO Kafka version: 4.1.0 (org.apache.kafka.common.uti
ls.AppInfoParser)
[2025-09-22 10:59:43,307] INFO Kafka commitId: 13f70256db3c994c (org.apache.kafk
a.common.utils.AppInfoParser)
[2025-09-22 10:59:43,307] INFO Kafka startTimeMs: 1758531583306 (org.apache.kafk
a.common.utils.AppInfoParser)
[2025-09-22 10:59:43,309] INFO [KafkaRaftServer nodeId=1] Kafka Server started (
kafka.server.KafkaRaftServer)
```

Kafka CLI



Kafka CLI

- Para facilitar el acceso desde cualquier sitio, en `.bashrc`, escribimos la siguiente línea:
 - `PATH="$PATH:~/kafka/kafka_2.13-3.9.0/bin"`
 - Sustituir por la ruta donde tengamos instalado Kafka.
 - Abrir una consola nueva y probar

```
jam@Dell4:~$ kafka-topics.sh
Create, delete, describe, or change a topic.
Option                                Description
-----                                -
--alter                                Alter the number
                                         replica assign
```

- Debería cargarse una serie de comandos, como se ve en la imagen.

Referencia

<https://kafka.apache.org/documentation/>

Kafka topics CLI

- Creación de topic
 - `kafka-topics.sh --create --topic first_topic --bootstrap-server 127.0.0.1:9092`
 - A esta línea podemos añadirle más comandos, por ejemplo
 - `kafka-topics.sh --create --topic second_topic --bootstrap-server 127.0.0.1:9092 --partitions 5`
- Ver lista de topics
 - `kafka-topics.sh --list --bootstrap-server 127.0.0.1:9092`
- Ver descripción de un topic
 - `kafka-topics.sh --describe --topic second_topic --bootstrap-server 127.0.0.1:9092`

```
jam@Dell14:~$ kafka-topics.sh --list --bootstrap-server 127.0.0.1:9092
first_topic
second_topic
```

```
jam@Dell14:~$ kafka-topics.sh --describe --topic second_topic --bootstrap-server 127.0.0.1:9092
Topic: second_topic      TopicId: 3_Xpl_YKQtuiFJBd8ZGr-Q PartitionCount: 5      ReplicationFactor: 1    Configs: segment.bytes=1073741824
Topic: second_topic      Partition: 0      Leader: 1      Replicas: 1      Isr: 1 Elr:      LastKnownElr:
Topic: second_topic      Partition: 1      Leader: 1      Replicas: 1      Isr: 1 Elr:      LastKnownElr:
Topic: second_topic      Partition: 2      Leader: 1      Replicas: 1      Isr: 1 Elr:      LastKnownElr:
Topic: second_topic      Partition: 3      Leader: 1      Replicas: 1      Isr: 1 Elr:      LastKnownElr:
Topic: second_topic      Partition: 4      Leader: 1      Replicas: 1      Isr: 1 Elr:      LastKnownElr:
```

- **Topic:** Nombre del topic.
- **TopicId:** Identificador único del topic.
- **Partition:** Número de partición del topic.
- **Leader:** Broker responsable de manejar todas las lecturas y escrituras para la partición.
- **Replicas:** Lista de brokers que almacenan una copia de la partición.
- **Isr (In-Sync Replicas):** Réplicas que están sincronizadas con el líder.

Kafka topics CLI

- Algunos topics no podremos crearlos, por ejemplo, topic con replication, no vamos a poder crearlo en un cluster con un solo broker, porque no puede poner la replica en un segundo broker.

```
jam@Dell4:~$ kafka-topics.sh --create --topic third_topic --bootstrap-server 127.0.0.1:9092 --partitions 3 --replication-factor 2
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.
Error while executing topic command : Unable to replicate the partition 2 time(s): The target replication factor of 2 cannot be reached because only 1 broker(s) are registered.
[2025-02-13 08:18:38,159] ERROR org.apache.kafka.common.errors.InvalidReplicationFactorException: Unable to replicate the partition 2 time(s): The target replication factor of 2 cannot be reached because only 1 broker(s) are registered.
(org.apache.kafka.tools.TopicCommand)
```

- Borrar un topic
 - `kafka-topics.sh --delete --topic second_topic --bootstrap-server localhost:9092`

Guía de nombrar topic

- Se recomienda un formato de este estilo:
 - <tipo de mensaje>.<nombre dataset>.<nombre del dato>.<formato del dato>
- Tipo de mensaje:
 - Logging, queiuing, tracking, etl, streaming, push, user

Topics

- Número ilimitado
- Definido por su nombre
- Cualquier formato de mensaje: json, avro, txt, binario...
- Los mensajes mandados terminan en las distintas particiones, con una id a partir de 0.
- Una vez un dato está escrito en una partición, no puede modificarse.



Topics

- El orden de los mensajes se garantiza en cada partición, no entre ellas.
- Si no damos una key, los mensajes se guardarán de siguiendo round robin entre particiones (partición0, partición1, partición 2, partición o...) , con una key, se asignarán a la misma partición.
- Ejemplo camiones de reparto.

Kafka console producer CLI



- Producir mensaje desde consola
- Producir mensaje con key

Kafka console producer CLI

- Para ver todos los comandos disponibles

- kafka-console-producer.sh

- Para enviar mensajes a un topic de Kafka desde la línea de comandos:

- kafka-console-producer.sh --bootstrap-server 127.0.0.1:9092 --topic first_topic

```
jam@Dell14:~/kafka$ kafka-console-producer.sh --bootstrap-server 127.0.0.1:9092 --topic first_topic
>Hola holita
>Me llamo Jon
>Jijiji
```

- Nos permite ir escribiendo mensajes a mano a un topic. Ctrl+C para salir.

- Esperar confirmación de todas las réplicas sincronizadas antes de considerar el mensaje como enviado con éxito.

- kafka-console-producer.sh --bootstrap-server 127.0.0.1:9092 --topic first_topic --producer-property acks=all

- Escribir a un topic inexistente → mensaje de error, pero crea el topic. Este funcionamiento puede cambiarse (`auto.create.topics.enable=false` en `server.properties`).

```
jam@Dell14:~/kafka$ kafka-console-producer.sh --bootstrap-server 127.0.0.1:9092 --topic prueba
>hola?
[2025-02-13 10:17:02,626] WARN [Producer clientId=console-producer] The metadata response from the cluster reported a recoverable issue with correlation id 5 : {prueba=UNKNOWN_TOPIC_OR_PARTITION} (org.apache.kafka.clients.NetworkClient)
>
^Cjam@Dell14:~/kafka$ kafka-topics.sh --list --bootstrap-server 127.0.0.1:9092
__consumer_offsets
first_topic
prueba
second topic
```

Por default se crea con una partición, puede cambiar esto en `server.properties`
`num.partitions=3`

Kafka console producer CLI

- Cada mensaje puede llevar asociada una key, que lo identifique.
- Para escribir mensajes con keys:

- `kafka-console-producer.sh --bootstrap-server 127.0.0.1:9092 --topic first_topic --property parse.key=true --property key.separator=":"`

```
user1 : Hola, soy el primer mensaje
user2 : Este es otro mensaje del usuario 2
user3 : Último mensaje de ejemplo
```

Al ver los mensajes del topic

```
null : Hola holita
null : Me llamo Jon
null : Jijiji
null : esto es otra prueba
null : auuuuuuuuu
null : learning non stop
User1 : Esto es una prueba
User2 : para ver la función key
User3 : con esto vale
```

Kafka console consumer CLI



- Consumir por el principio.
- Consumir por el final.
- Mostrar key y valor.
- Parámetro grupo

Kafka console consumer CLI

- Ver las opciones del comando
 - kafka-console-consumer.sh
- Consumir por el principio
 - kafka-console-consumer.sh --bootstrap-server 127.0.0.1:9092 --topic first_topic --from-beginning
- Consumir en tiempo real. Si no especificamos nada, nos aparecerán los mensajes que se van escribiendo en el momento concreto.

```
jam@De114:~/kafka$ kafka-consoconsumer.sh --bootstrap-server  
nt.value=true --property key.separator=" : " --from-beginning  
null : Hola holita  
null : Me llamo Jon  
null : Jijiji  
null : esto es otra prueba  
null : auuuuuuuuu  
null : learning non stop  
User1 : Esto es una prueba  
User2 : para ver la función key  
User3 : con esto vale
```

- En una terminal: kafka-console-consumer.sh --bootstrap-server 127.0.0.1:9092 --topic first_topic
- En otra terminal, escribimos mensajes (console producer)
- Veremos que no aparece nada hasta escribirlo en la Segunda terminal.

```
jam@De114:~/kafka$ kafka-console-consumer.sh --bootstrap-server 127.0.0.1:9092 --topic first_topic  
probando en tiempo real
```

- Ver mensajes con key y mensaje.
 - kafka-console-consumer.sh --bootstrap-server 127.0.0.1:9092 --topic first_topic --property print.key=true --property print.value=true --property key.separator=" : " --from-beginning

```
null : Hola holita  
null : Me llamo Jon  
null : Jijiji  
null : esto es otra prueba  
null : auuuuuuuuu  
null : learning non stop  
User1 : Esto es una prueba  
User2 : para ver la función key  
User3 : con esto vale  
null : probando en tiempo real
```

Producer y consumer básicos

- Los archivos producer.py y consumer.py son scripts básicos para mandar un mensaje y leer en un topic.

```
def main():
    log.info("Soy un Kafka Producer")
    bootstrap_servers = "127.0.0.1:9092"
    producer = KafkaProducer(
        bootstrap_servers=bootstrap_servers,
        key_serializer=lambda k: k.encode('utf-8') if k is not None else None,
        value_serializer=lambda v: v.encode('utf-8') if v is not None else None
    )
    topic = "demo_python"
    message = "hello world"
    producer.send(topic, key="prueba", value=message)
    producer.flush()
    producer.close()
```

Se conecta al servidor de kafka (en local, puerto 9092).
Escribe en el topic demo_Python, el mensaje "hello world", con la key "prueba".

*pip install kafka-python para poder lanzar los scripts

Producer y consumer básicos

- Lee los mensajes en demo_python, mostrando key y value.

```
def main():
    consumer = KafkaConsumer(
        'demo_python',
        bootstrap_servers='127.0.0.1:9092',
        auto_offset_reset='earliest',
        key_deserializer=lambda k: k.decode('utf-8') if k is not None else None,
        value_deserializer=lambda v: v.decode('utf-8') if v is not None else None,
        group_id='demo_group'
    )
    for message in consumer:
        print("Key:", message.key, "Value:", message.value)
```


Pruebas

- ¿Qué pasa cuando no hay consumer?
- ¿Qué pasa cuando el producer está emitiendo y levantamos el consumer en cualquier momento?
- ¿Qué pasa cuando hay 2 consumidores a la vez leyendo del mismo topic?
- Modificar el producer para que lance números al azar entre el 1 y el 100 cada 3 segundos, para ver el funcionamiento de continuo.

Kafka console consumer CLI

- **Consumer groups**

- Un **consumer group** es un conjunto de consumidores que colaboran para leer mensajes de un topic de forma coordinada. Cada mensaje es procesado por **uno y solo uno de los consumidores del grupo**.
- **Cómo se distribuyen los mensajes:**
- **Número de particiones = Número de consumidores:** Cada consumidor procesa una partición.
- **Número de particiones > Número de consumidores:** Algunos consumidores procesarán varias particiones.
- **Número de consumidores > Número de particiones:** Algunos consumidores quedarán inactivos (no recibirán mensajes).
- Vamos a crear un topic nuevo (third_topic) con 3 particiones para empezar de cero.

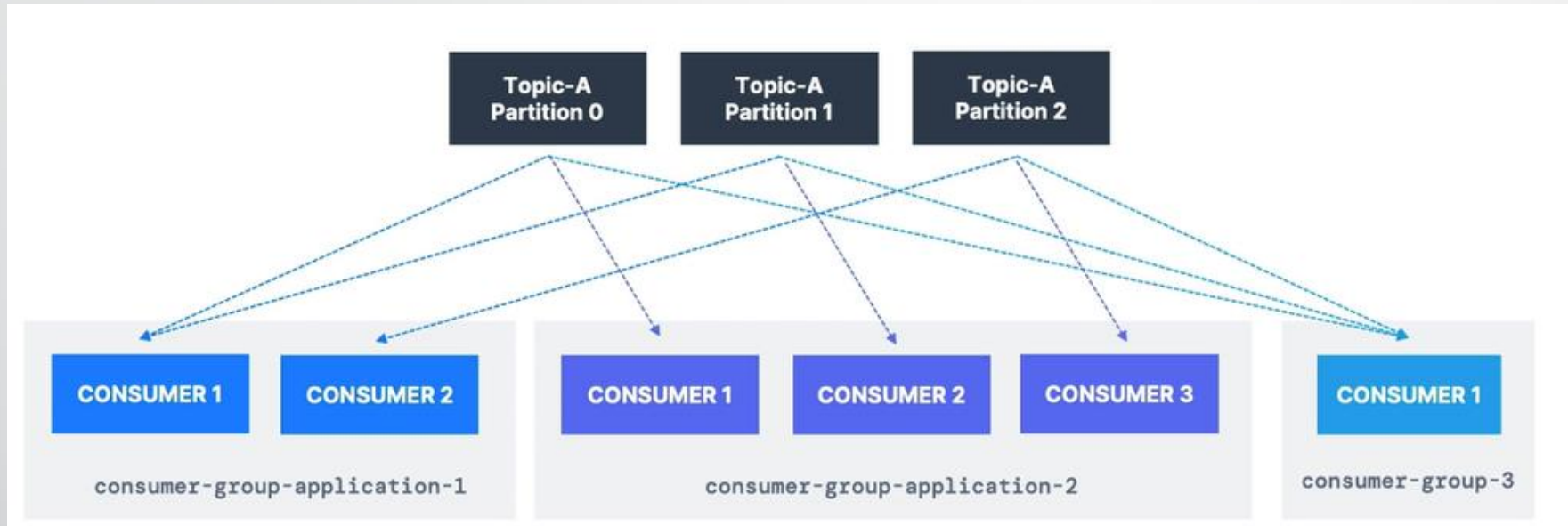
- **Lanzamos**

- `kafka-console-consumer.sh --bootstrap-server 127.0.0.1:9092 --topic third_topic --group my-first-app`
- `kafka-console-producer.sh --bootstrap-server 127.0.0.1:9092 --topic third_topic --producer-property partitioner.class=org.apache.kafka.clients.producer.RoundRobinPartitioner` (así cada mensaje va a una partición).
- Vamos creando mensajes y veremos que el consumer coge todo.
- Si lanzamos en una tercera terminal de nuevo el consumer “my-first-app”, veremos que los mensajes se dividen entre los dos.
- Si lanzamos otro grupo distinto “my-second-app”, funcionará a parte de “my-first-app”, leyendo todos los mensajes del topic.

Kafka console consumer CLI

- Hacer pruebas con
 - Quitar roundrobin del productor y poner key
 - `kafka-console-producer.sh --bootstrap-server 127.0.0.1:9092 --topic third_topic --property parse.key=true --property key.separator=":"`
 - Lanzar mensajes con user1: x, user2:y y ver cómo se reparten.

Kafka consumer groups CLI



- Listar grupos
- Describir un grupo

Kafka consumer groups CLI

- Lista de grupos de consumo

- `kafka-consumer-groups.sh --bootstrap-server localhost:9092 --list`

```
jam@Dell14:~/kafka$ kafka-consumer-groups.sh --bootstrap-server localhost:9092 --list
my-first-app
my-second-app
```

- Describir un grupo

- `kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --group my-first-app`

```
jam@Dell14:~/kafka$ kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --group my-first-app
```

GROUP	TOPIC	PARTITION	CURRENT-OFFSET	LOG-END-OFFSET	LAG	CONSUMER-ID
HOST	CLIENT-ID					
my-first-app	third_topic	1	5	5	0	console-consumer-41
9376a /127.0.0.1	console-consumer					
my-first-app	third_topic	0	6	6	0	console-consumer-41
9376a /127.0.0.1	console-consumer					
my-first-app	third topic	2	6	6	0	console-consumer-5e

- TOPIC: Nombre del topic que el grupo está consumiendo (third_topic).
 - PARTITION: Número de la partición del topic asignada al consumidor.
 - CURRENT-OFFSET: Último mensaje que el consumidor ha leído en esa partición.
 - LOG-END-OFFSET: Última posición del log en la partición (el mensaje más reciente).
 - LAG: Diferencia entre LOG-END-OFFSET y CURRENT-OFFSET, indica cuántos mensajes pendientes tiene el consumidor.
 - CONSUMER-ID: Identificador del consumidor dentro del grupo (console-consumer-41...).

Consumer groups en python

- Vamos a utilizar de base el script kafka-producer-3keys.py y kafka-consumer-3keys.py
- Vamos a borrar el topic demo_python y vamos a crearlo con 3 particiones.
- Lanzamos el producer y lanzamos una vez el consumer, vemos qué pasa.
- Lanzamos en otra terminal un segundo consumer. ¿Qué pasa? ¿Cómo se dividen los mensajes?

```
(bda) jam@Dell14:~/kafka/kafka-python$ python3 consumer-3keys.py
[Consumer Dell14_8102] Partición: 2, Key: key1, Value: 32
[Consumer Dell14_8102] Partición: 2, Key: key1, Value: 17
[Consumer Dell14_8102] Partición: 2, Key: key2, Value: 76
[Consumer Dell14_8102] Partición: 2, Key: key1, Value: 6
[Consumer Dell14_8102] Partición: 2, Key: key2, Value: 21
[Consumer Dell14_8102] Partición: 2, Key: key1, Value: 89
[Consumer Dell14_8102] Partición: 2, Key: key2, Value: 40
[Consumer Dell14_8102] Partición: 2, Key: key1, Value: 62
```

```
(bda) jam@Dell14:~/kafka/kafka-python$ python3 consumer-3keys.py
[Consumer Dell14_2205] Partición: 1, Key: key3, Value: 18
[Consumer Dell14_2205] Partición: 1, Key: key3, Value: 44
[Consumer Dell14_2205] Partición: 1, Key: key3, Value: 84
[Consumer Dell14_2205] Partición: 1, Key: key3, Value: 74
[Consumer Dell14_2205] Partición: 1, Key: key3, Value: 4
[Consumer Dell14_2205] Partición: 1, Key: key3, Value: 100
```

Kafka consumer groups CLI – Reset offsets



- Iniciar/apagar un console consumer
- Resetear offsets
- Empezar el console consumer y ver qué ocurre.

Kafka consumer groups CLI – Reset offsets

- Describir el grupo de consumo
 - `kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --group my-first-app`
- Dry Run: reset los offsets al principio de cada partición
 - `kafka-consumer-groups.sh --bootstrap-server localhost:9092 --group my-first-app --reset-offsets --to-earliest --topic third_topic --dry-run`
- Ejecutar el offset
 - `kafka-consumer-groups.sh --bootstrap-server localhost:9092 --group my-first-app --reset-offsets --to-earliest --topic third_topic --execute`
- Volver a describir
 - `kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --group my-first-app`
- Consumir desde donde se ha hecho el reset
 - `kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic third_topic --group my-first-app`

Eligiendo número de particiones y
factor de replicación



Nº de particiones

- Cada partición puede gestionar unos pocos MB/s
- Más particiones:
 - Mejor paralelismo, mejor rendimiento.
 - Posibilidad de tener más consumidores en un grupo para escalar.
 - Poder lanzar más brokers si tenemos un cluster grande.
 - Mayor latencia.
- Guía:
 - Cluster pequeño (< 6 brokers): 3 x nº brokers
 - Cluster grande (> 12 brokers): 2 x nº de brokers
 - Ajustar el nº de consumidores que necesitamos corriendo en paralelo en el pico de rendimiento.

Factor de replicación

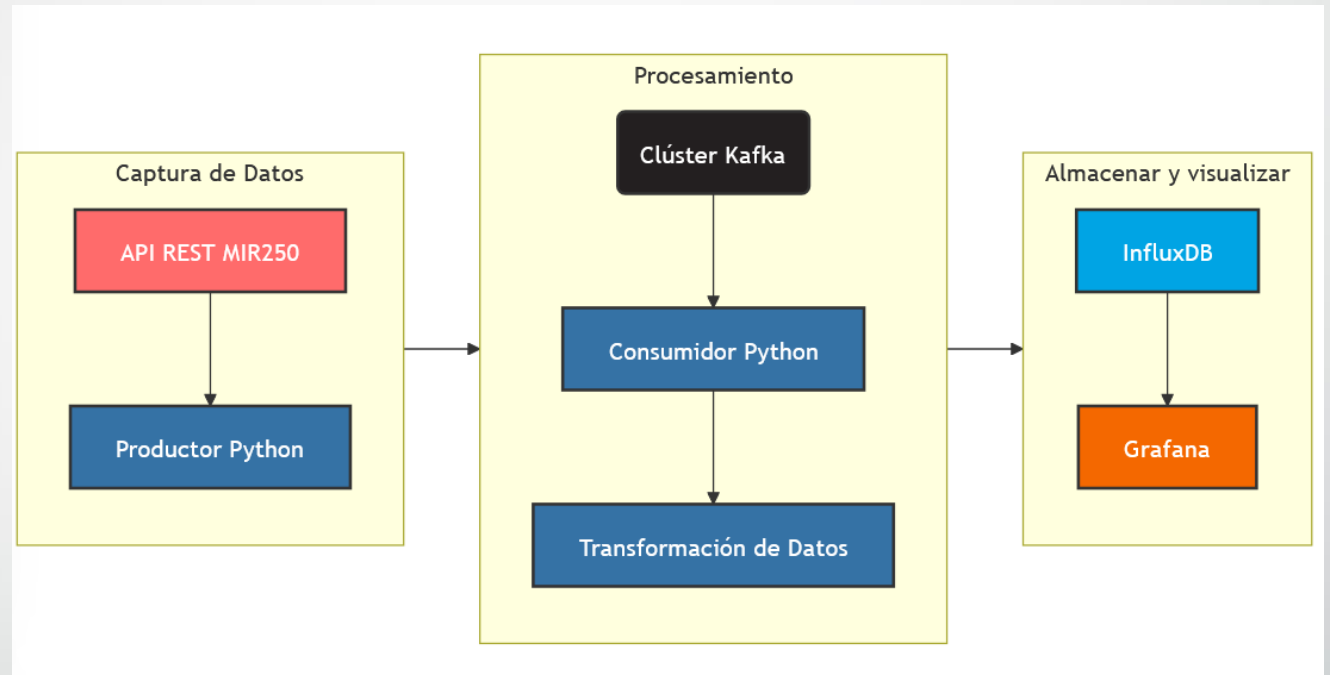
- Debería ser mínimo 2, normalmente es 3, máximo 4.
- Mayor factor de replicación (N).
 - Más durabilidad del sistema (N-1 brokers pueden fallar).
 - Mayor latencia (si acks=all)
 - Mayor espacio necesario en disco.

Programación python Kafka



Creación de un flujo para MIR250

Flujo de trabajo para obtener información del MIR250, mandar a server de Kafka, procesarla y subirla a influxdb + grafana



Pasos

- Creación del topic
- Lanzar el producer que se comunica con la API
- Creación del bucket en influxdb
- Lanzar el consumidor
- Graficar el grafana

Configuración del productor Wikimedia producer

```
conf = {  
  # Conexión  
  'bootstrap.servers': 'localhost:9092',  
  'client.id': 'wikimedia-producer',  
  
  # Confiabilidad  
  'acks': 'all',           # Máxima durabilidad  
  'retries': 5,           # Reintentos limitados  
  
  # Rendimiento  
  'compression.type': 'snappy', # Eficiente para datos similares/repetitivos  
  'linger.ms': 100,        # Acumulación para mejor eficiencia de red  
  'batch.size': 65536,     # Lotes más grandes (64KB)  
}
```

Valores Básicos

Configuración	Predeterminado	Recomendado	Justificación
bootstrap.servers	-	localhost:9092	Conexión al broker local
client.id	-	wikimedia-producer	Identificación del cliente en logs y métricas

Configuración del productor Wikimedia producer

```
conf = {  
  # Conexión  
  'bootstrap.servers': 'localhost:9092',  
  'client.id': 'wikimedia-producer',  
  
  # Confiabilidad  
  'acks': 'all',           # Máxima durabilidad  
  'retries': 5,           # Reintentos limitados  
  
  # Rendimiento  
  'compression.type': 'snappy', # Eficiente para datos similares/repetitivos  
  'linger.ms': 100,         # Acumulación para mejor eficiencia de red  
  'batch.size': 65536,     # Lotes más grandes (64KB)  
}
```

Confiabilidad

Configuración	Predeterminado	Recomendado	Justificación
acks	1	all	Garantiza que los datos se replican completamente antes de confirmar
retries	2147483647	5	Número razonable de reintentos en caso de fallos

Configuración del productor Wikimedia producer

```
conf = {
  # Conexión
  'bootstrap.servers': 'localhost:9092',
  'client.id': 'wikimedia-producer',

  # Confiabilidad
  'acks': 'all',           # Máxima durabilidad
  'retries': 5,           # Reintentos limitados

  # Rendimiento
  'compression.type': 'snappy', # Eficiente para datos similares/repetitivos
  'linger.ms': 100,         # Acumulación para mejor eficiencia de red
  'batch.size': 65536,      # Lotes más grandes (64KB)
}
```

Rendimiento y Eficiencia

Configuración	Predeterminado	Recomendado	Justificación
compression.type	none	snappy	Buen equilibrio entre velocidad y ratio de compresión para datos JSON
linger.ms	0	100	Acumula mensajes 100ms para reducir overhead de red
batch.size	16384	65536	Lotes de 64KB para mejorar throughput y compresión
buffer.memory	33554432	33554432	Valor predeterminado adecuado (32MB)

Configuración del producer

Idempotencia

- La idempotencia en Kafka es una propiedad que garantiza que los mensajes enviados por un productor se escriban exactamente una vez en el log, incluso si el productor reintentra el envío varias veces debido a errores de red o timeouts.
- Cuando un productor tiene habilitada la idempotencia:
 - Se le asigna un ID único (PID)
 - Cada mensaje lleva un número de secuencia
 - El broker mantiene un registro de mensajes ya procesados
 - Los duplicados son detectados y descartados automáticamente

```
props.put("enable.idempotence", true)
```

Configuración del consumer

Delivery semantics

Kafka admite tres semánticas de entrega principales:

1. Como máximo una vez (At most once): Los mensajes pueden perderse pero nunca duplicarse. El consumidor confirma los offsets inmediatamente después de recibirlos, antes de procesarlos. Menor latencia, sin garantías de procesamiento.

2. Al menos una vez (At least once): Los mensajes nunca se pierden pero pueden duplicarse. El consumidor confirma los offsets solo después de un procesamiento exitoso. Este es el comportamiento predeterminado de Kafka. Requiere operaciones idempotentes para manejar duplicados.

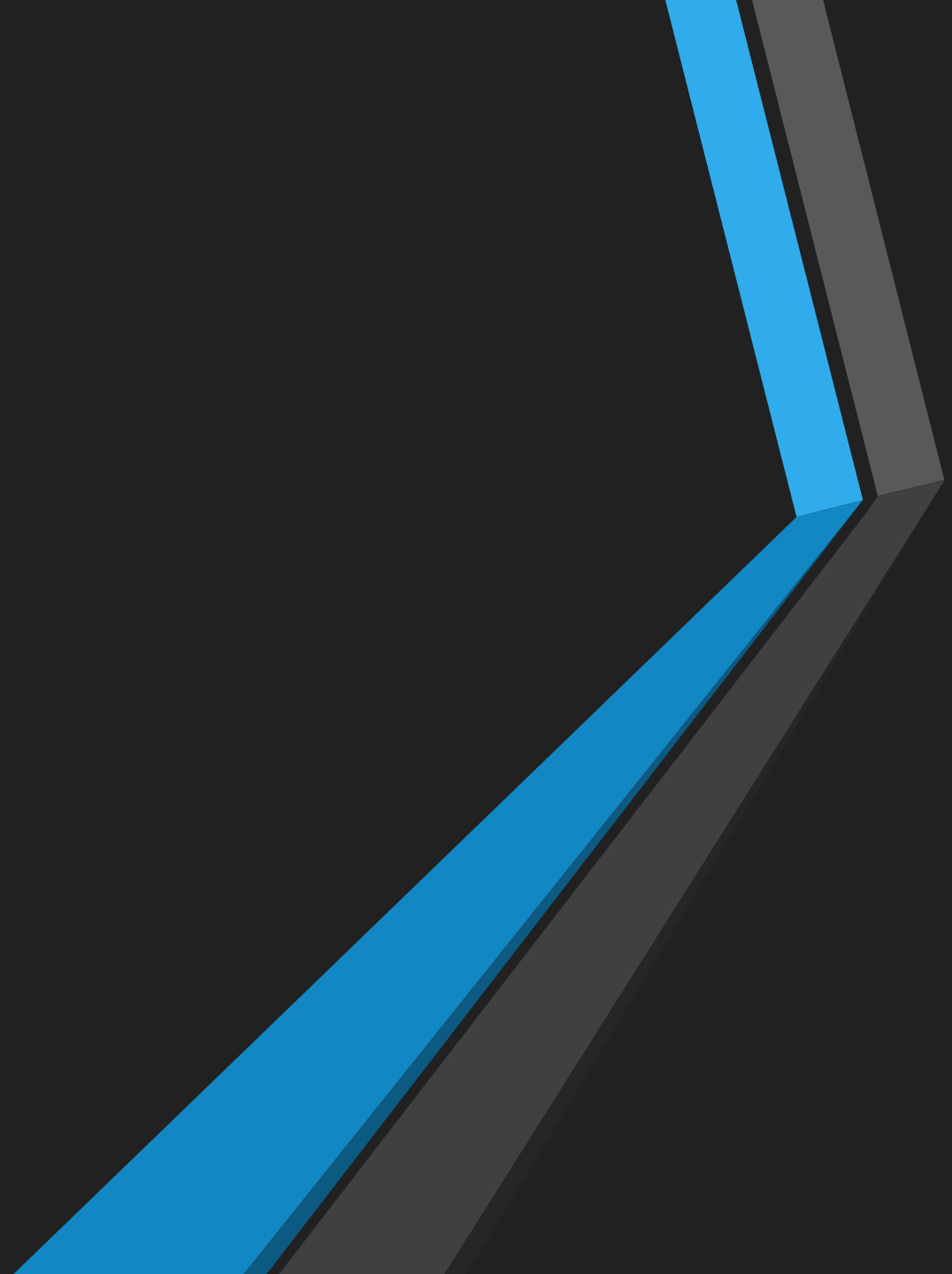
3. Exactamente una vez (Exactly once): Los mensajes se entregan exactamente una vez sin pérdidas ni duplicación. Se implementa mediante:

- Productores idempotentes (`enable.idempotence=true`)
- API transaccional para escrituras atómicas entre particiones/temas
- Marcadores de transacciones y niveles de aislamiento para consumidores

Tu consumidor actual utiliza semánticas de al menos una vez al configurar

`enable.auto.commit: False` y confirmar manualmente después del procesamiento.

Kafka Connectors



Kafka Connectors

Definición

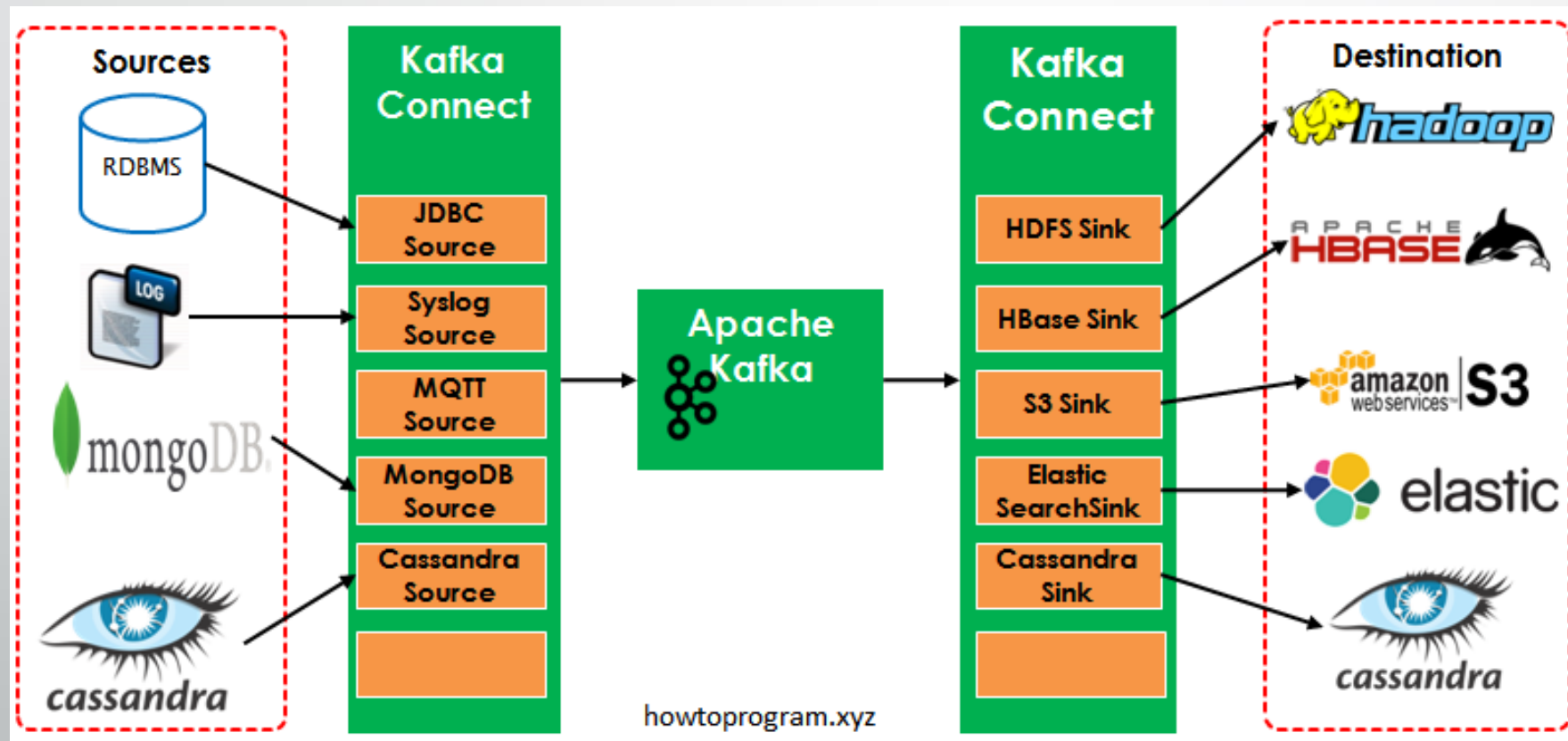
Los Kafka Connectors son componentes del ecosistema de Apache Kafka que facilitan la conexión entre Kafka y otros sistemas de datos, permitiendo importar datos hacia Kafka (source connectors) o exportar desde Kafka hacia otros sistemas (sink connectors).

Ejemplos:

- S3 Connector: Permite guardar datos de Kafka en Amazon S3
- Elasticsearch Connector: Exporta datos de Kafka a Elasticsearch para búsquedas
- MongoDB Connector: Sincroniza datos entre MongoDB y Kafka

Kafka Connectors

Ejemplo gráfico



Kafka Connectors

Ventajas

Los Kafka Connectors ofrecen varias ventajas importantes:

- 1. Eliminan código personalizado:** No necesitas desarrollar y mantener código propio para integrar sistemas.
- 2. Configuración declarativa:** Se configuran con archivos JSON en lugar de escribir código.
- 3. Escalabilidad automática:** Pueden distribuir el trabajo entre múltiples tareas de forma transparente.
- 4. Tolerancia a fallos:** Gestionan automáticamente reconexiones y reintentos.
- 5. Gestión de offsets:** Controlan exactamente qué datos se han procesado para garantizar entrega exactly-once.
- 6. Transformaciones incorporadas:** Incluyen funcionalidades para modificar datos en tránsito sin código adicional.
- 7. Monitorización centralizada:** Proporcionan métricas y estadísticas para supervisar el rendimiento.
- 8. Comunidad activa:** Existe un amplio ecosistema de conectores para muchos sistemas.
- 9. Reducción de latencia:** Optimizados para alto rendimiento en comparación con soluciones personalizadas.
- 10. Menor carga operativa:** Reducen el trabajo de DevOps al unificar la gestión de integraciones.

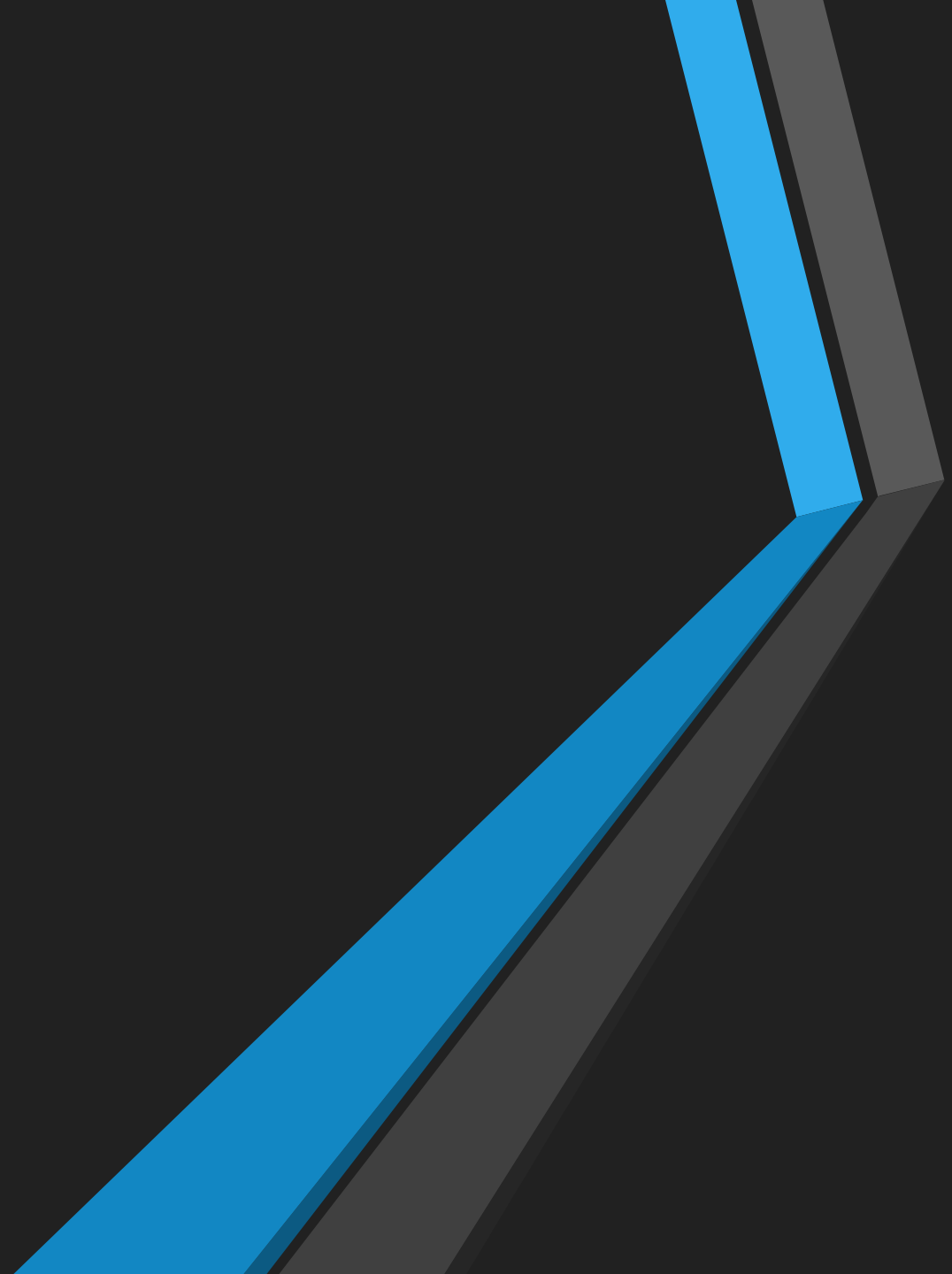
Source connectors vs Sink connectors

- **Source Connectors:** traen datos DE sistemas externos A Kafka (ej: MySQL → Kafka)
- **Sink Connectors:** llevan datos DE Kafka A sistemas externos (ej: Kafka → Elasticsearch)

Ejemplo:

En vez de programar código para leer una base de datos y publicar en Kafka, usas un conector MySQL Source que lo hace automáticamente. Solo configuras (JSON) qué tabla leer, cada cuánto, etc.

Estudio de casos

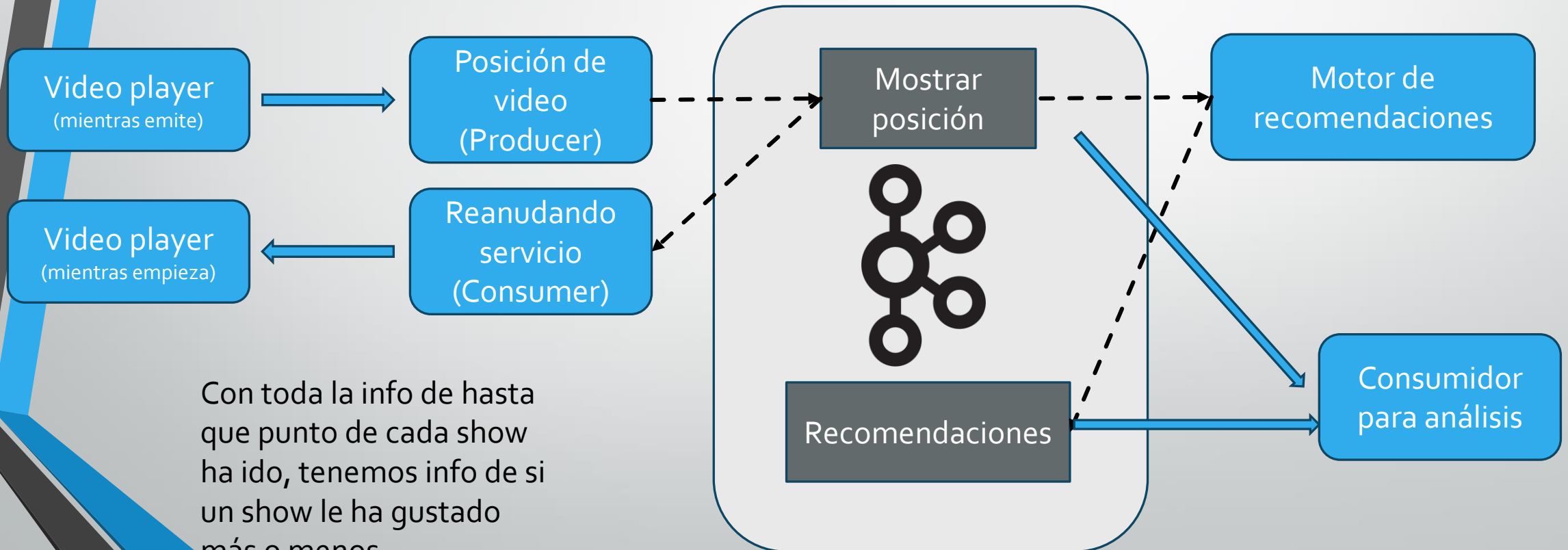


Netflix

- Asegurarte que puedes volver a iniciar el video donde lo has dejado.
- Construir el perfil de usuario en tiempo real
- Recomendar la siguiente serie al usuario en tiempo real
- Guardar todos los datos en bases para análisis.

Arquitectura - Netflix

- Tendremos un topic que sea “Mostrar posición”, que diga en qué punto se han quedado los usuarios en un programa concreto.
- No necesitamos conocer cada punto de reanudación, sólo el último de cada programa.



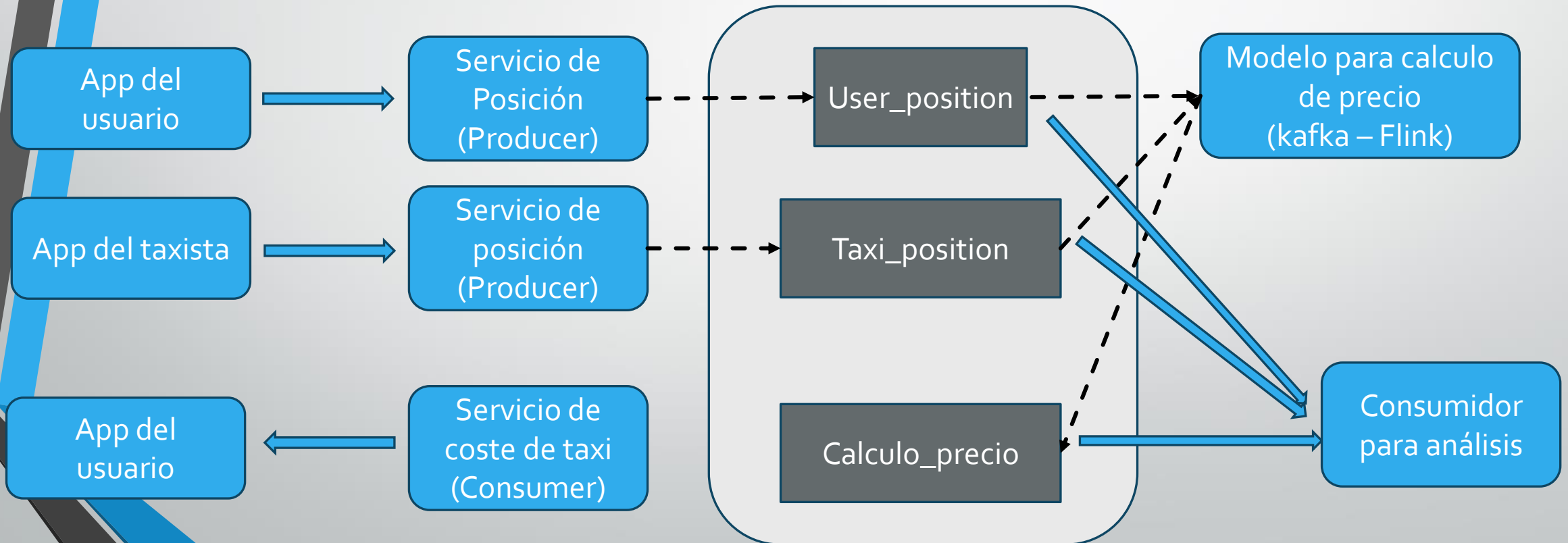
Arquitectura - Netflix

- Topic “mostrar_posicion”
 - Va a tener múltiples productores
 - Si tenemos que elegir una clave → “user_id”
 - Tráfico muy elevado
- Topic “recomendaciones”
 - Tráfico más reducido (no hace recomendaciones distintas cada 30 segundos)
 - Seguramente la key también sería “user_id”

Uber

- Compañía para coger taxi bajo demanda al momento.
 - El usuario tiene que matchear con un conductor relativamente cercano.
 - El precio varía según número de conductores y número de pasajeros.
 - Todos los datos de posición antes y durante el recorrido tienen que guardarse para poder computar el coste correctamente.

Arquitectura - Uber



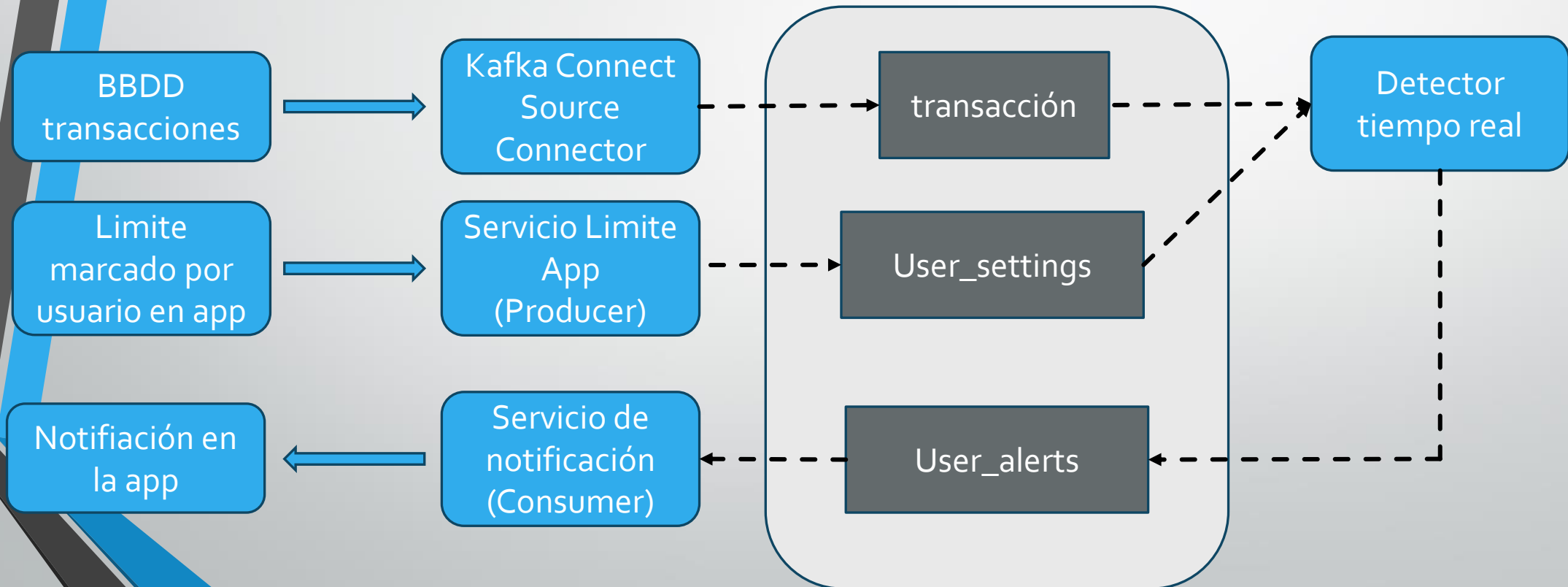
Arquitectura - Uber

- Los topics de posición:
 - Tienen múltiples productores → alta distribución durante momentos de alto volumen
 - La key sería "user_id", "taxi_id".
 - Los datos no hay que guardarlos demasiado tiempo en el cluster, ya que varían cada poco tiempo.

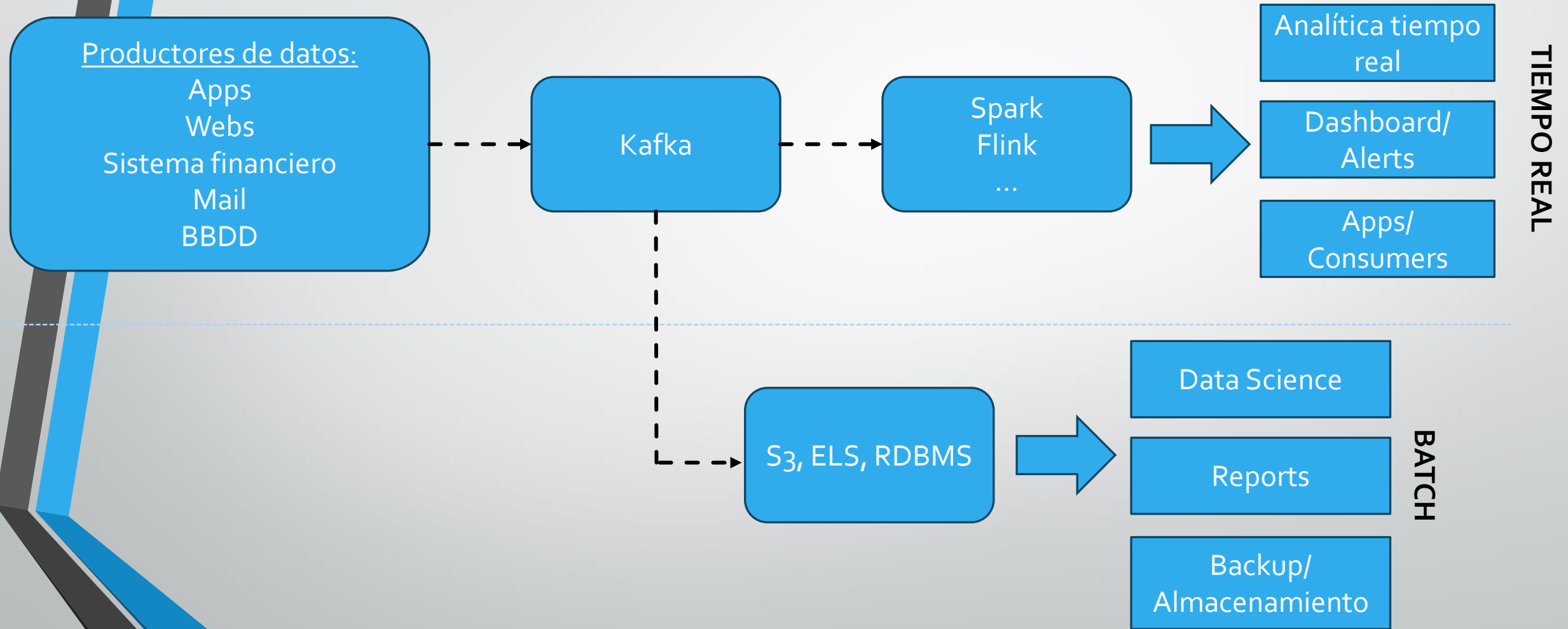
Banco

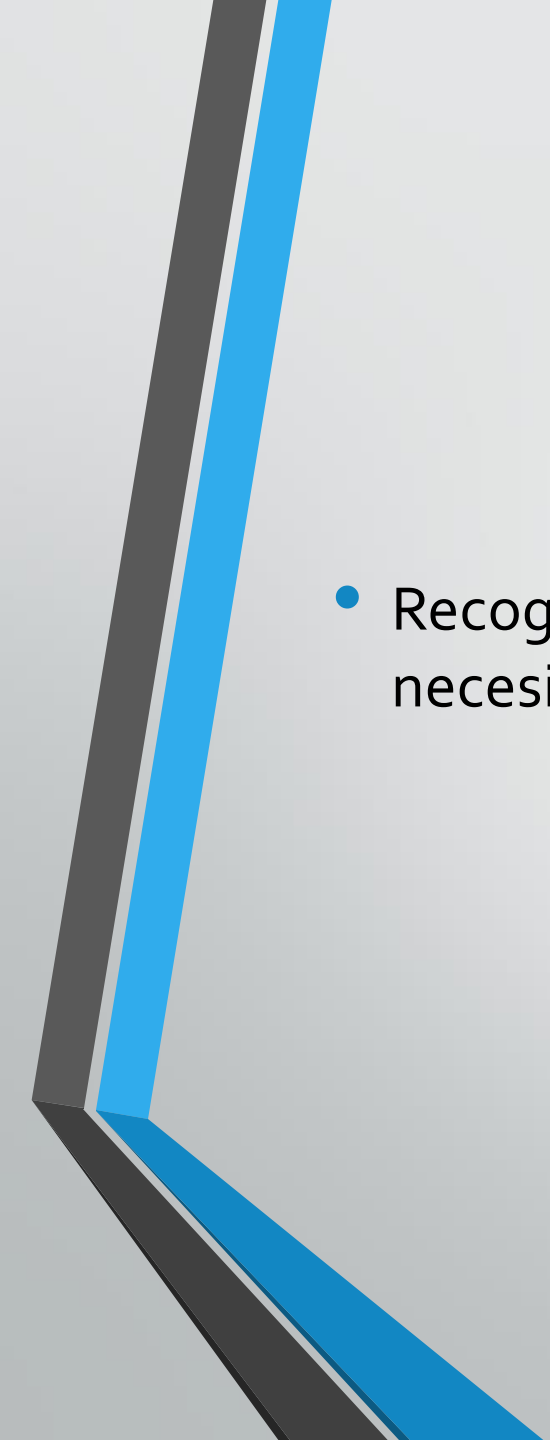
- Imaginemos que queremos desarrollar una alerta para los usuarios en caso de transacciones grandes.
 - El dato de la transacción ya está en la BBDD.
 - El límite de dinero puede ser definido por el cliente.
 - Las alertas tienen que mandarse en tiempo real a los usuarios.

Arquitectura - Banco



Ingesta Big Data





Logs y métricas

- Recoger logs y métricas de todas las aplicaciones y mandarlas a donde necesitemos.